

<pre>#include <iostream> // Standard library #include "myfile.h" // User-defined (protect w/ #ifndef) // Static: Allocated at compile-time, exists for program duration. int global_var; // Static Memory Stack: Function-local vars, auto-allocated/deallocated (LIFO). -> Heap: Dynamic allocation (new/delete).</pre>	<pre>#ifndef MYFILE_H #define MYFILE_H // Code #endif void func() { int x; }; int* p = new int(5); delete p; // Free memor</pre>	Memory Segments: Static, Stack, Heap <pre>c++ hello.cpp -o hello -std=c++14 // Compile & link c++ -c hello.cpp -o hello.o // Compile separately c++ hello.o -o hello // Link separately ./hello // Execute c++ -c -g hello.cpp -o hello.o -std=c++14 // Debug int* p = new int[5]; // Alloc array delete[] p; // De</pre>	<pre>#include <iostream> int main() { std::cout << "Hello world!" << std::endl; return 0; }; int* arr = new int[10]; // Allocate delete[] arr; // Deallocate</pre>	Top-Down Approach Problem broken in smaller parts Structured programing languages w/ documn. of module and debu no comm. Between modules contains redundant info decomposition approach	Bottom Up Approach Smaller problems are solved Object oriented programing Used in testing modules Yes commn. Bet. Modules No redundant info Composition approach																																																																																						
<pre>int x = 10; int* p = &x; // Store address of x *p = 20; // Change x to 20 cout << *p; // Dereference</pre>	<pre>int arr[] = {1, 2, 3}; int* p = arr; p++; // Moves to next element cout << *(p+1); // Access arr[2]</pre>	<pre>int x = 10; int* p = &x; int** pp = &p; cout << **pp; // 10</pre>	<pre>const int* ptr; // Pointer to const int (can't modify value) int* const ptr; // Const pointer (can't change address) void print(const std::string& s) { std::cout << s; } // Const const int* const ptr; // Both const pass by referenc:</pre>	<pre>int x = 10; int& ref = x; ref = 20; // x becomes 20 void modify(int& x) { x += 10; }</pre>	<table><tr><td>Feature</td><td>Reference</td><td>Pointer</td></tr><tr><td>Null?</td><td>No</td><td>Yes</td></tr><tr><td>Reassignment?</td><td>No</td><td>Yes</td></tr><tr><td>Dereferencing?</td><td>Implicit</td><td>Explicit *</td></tr></table>	Feature	Reference	Pointer	Null?	No	Yes	Reassignment?	No	Yes	Dereferencing?	Implicit	Explicit *																																																																										
Feature	Reference	Pointer																																																																																									
Null?	No	Yes																																																																																									
Reassignment?	No	Yes																																																																																									
Dereferencing?	Implicit	Explicit *																																																																																									
NAMESPACE namespace MyNamespace { int x = 10; } //uses: std::cout << MyNamespace::x; // Fully qualified using namespace MyNamespace; // Global use using MyNamespace::x; // Local use	<pre>std::string s = "Hello"; s += " World"; // Concat s.length(); // Get length s.substr(0,3); // Substring s.find("lo"); // Find index</pre>	<pre>int arr[5] = {1, 2, 3, 4, 5}; // Fixed-size array (last char is \0) arr[10] = 99; //X! Undefined behavior (no bounds checking!) char(1),short(2),int(4),long(4/8),float(4),double(8) numerical types, both signed and unsigned, als bool (like char) int** pp = &p; // Pointer to pointer</pre>	<pre>static_cast<T>(expr) //Compile-time cast dynamic_cast<T>(ptr) //Runtime, for polymorphism const_cast<T>(expr) //Removes const reinterpret_cast<T>(expr) // Dangerous, type conversion int x = 10; double y = static_cast<double>(x);</pre>	CONTROL FLOWS if (x > 0) {...} for (int i = 0; i < n; i++) {...} while (x > 0) {...} switch (x) { case 1: break; default: break; }																																																																																							
SPIRAL RULE EXAMPLES char* str[10]; // Array of 10 pointers to char int (*fp)(int); // ptr to funcnt taking int, ret int int* (*pf)(); // ptr to function ret pointer to int int (*arr[5])(); // Array of 5 function pointers	RANGE LOOP std::vector<int> v = {1, 2, 3}; for (int x : v) { std::cout << x; } // Iterate over values for (auto& x : v) { x *= 2; } // Modify elements	TRY AND CATCH ERROR HANDLIGN try { (some function here, if not) throw std::runtime_error("Error!"); } // Report an error } catch (const std::exception& e) { //expect excptn. inpt std::cout << e.what(); } // Handle error	AUTO auto x = 42; // int auto d = 3.14; // double auto str = "Hello"; // const char* auto sum(int a, int b) -> int { return a + b; }	STRUCT VS CLASS <table><tr><td>Feature</td><td>Struct</td><td>Class</td></tr><tr><td>Deault Access</td><td>Public</td><td>Private</td></tr><tr><td>OOP Features</td><td>Limited</td><td>Full</td></tr><tr><td>C++ is class based, Obj-C is object</td><td></td><td></td></tr></table>	Feature	Struct	Class	Deault Access	Public	Private	OOP Features	Limited	Full	C++ is class based, Obj-C is object																																																																													
Feature	Struct	Class																																																																																									
Deault Access	Public	Private																																																																																									
OOP Features	Limited	Full																																																																																									
C++ is class based, Obj-C is object																																																																																											
<pre>std::vector<int> v = {1, 2, 3}; for (std::vector<int>::iterator it = v.begin(); it != v.end(); ++it) { std::cout << *it; // standard iterator } for (std::vector<int>::reverse_iterator rit = v.rbegin(); rit != v.rend(); ++rit) { std::cout << *rit; // Iterate backward } for (auto it = v.begin(); it != v.end(); ++it) { std::cout << *it; // auto iterators, cleaner }</pre>	ITERATORS	Inheritance Type : Base → Derived Access Public: public → public, protected → protected Protected: public → protected, protected → private Private: public/protected → private class Parent { protected: int a; }; class Child : public Parent { } // a remains protected class Child2 : private Parent { } // a becomes private	move forward: the ++ operator! A way read at the current index: the * operator test for the end: by comparing to an iterator set at end jump to any rank: with arithmetic on iterators! write at the current index: the * operator as a value "end" operator pointing past last element Begin pointing to beginning	ITERATORS	<pre>class A { // Overloading public: void func(int x); void func(double x); // Different parameter types class B : public A { // Overriding public: void func(int x) override; // Same function signatu</pre>																																																																																						
<pre>class Student { // lifetime, birth and death with const and dest public: Student() { std::cout << "Constructor!"; } // Default constructor Student(const Student& s) { name = s.name; } // Copy constructor Student(Student&& s) { name = std::move(s.name); } // Move constructo ~Student() { std::cout << "Destructor!"; } virtual void draw() = 0; // Must be overridden in derived class };</pre>	OVERWRITE	<pre>class Base { public: virtual void show() { std::cout << "Base"; }; class Derived : public Base { public: void show() override { std::cout << "Derived"; }; Base* b = new Derived(); b->show(); // Calls Derived::show() due to virtual</pre>	DESTRUCTOR	<table><tr><td>Feature</td><td>Overloading (same function name)</td><td>Overriding</td></tr><tr><td>Definition</td><td>different params</td><td>Redefining base class function</td></tr><tr><td>Polymorphism</td><td>Compile-time</td><td>Runtime</td></tr><tr><td>Scope</td><td>Same class</td><td>Derived class</td></tr></table> <pre>class A { virtual void func() final; // Can't be overridden class FinalClass final { // Can't be inherited</pre>	Feature	Overloading (same function name)	Overriding	Definition	different params	Redefining base class function	Polymorphism	Compile-time	Runtime	Scope	Same class	Derived class																																																																											
Feature	Overloading (same function name)	Overriding																																																																																									
Definition	different params	Redefining base class function																																																																																									
Polymorphism	Compile-time	Runtime																																																																																									
Scope	Same class	Derived class																																																																																									
<pre>class Student { //copies data frm one object to another public: Student& operator=(const Student& s) { // Copy assignment name = s.name; // Deep copy return *this; } };</pre>	<pre>class Student { // transfers ownership of resources public: Student& operator=(Student&& s) { // Move assignment name = std::move(s.name); // Avoids copying return *this; } };</pre>	<pre>class A { public: void show() { std::cout << "A"; }; class B : public A {}; class C : public A {}; // diamond problem class D : public B, public C {}; // ✗ Causes ambiguity</pre>	<pre>class A { public: void show() { std::cout << "A"; }; class B : virtual public A {}; class C : virtual public A {}; class D : public B, public C {}; // ✔ Resolves ambiguity</pre>	MULTIPLE INHERITENCE																																																																																							
Sequence Containers: implement data accessed in a sequential manner. Vector, list, deque, arrays, forward_list	<pre>std::vector<int> v = {1, 2, 3}; v.push_back(4); // Add element v.pop_back(); // Remove last elem</pre>	<pre>std::list<int> lst = {1, 2, 3}; lst.push_front(0); // Insert at front lst.push_back(4); // Insert at back</pre>	<pre>std::map<std::string, int> ages; ages["Alice"] = 25; ages["Bob"] = 30;</pre>	Container Adaptors: provide a different interface for sequential containers. Queue, priority_queue, stack	Associative Containers: implement sorted data structures that can be quickly searched (O(log n) Set, multiset, map, multimap																																																																																						
<table><tr><th>Metrics</th><th>C</th><th>C++</th><th>Java</th></tr><tr><td>Programming Paradigm</td><td>Procedural language</td><td>Object-Oriented Programming (OOP)</td><td>Pure Object Oriented</td></tr><tr><td>Origin</td><td>Based on assembly language</td><td>Based on C language</td><td>Based on C and C++</td></tr><tr><td>Developer</td><td>Dennis Ritchie in 1972</td><td>Bjarne Stroustrup in 1979</td><td>James Gosling in 1991</td></tr><tr><td>Translator</td><td>Compiler only</td><td>Compiler only</td><td>Interpreted language (Compiler + interpreter)</td></tr><tr><td>Platform Dependency</td><td>Platform Dependent</td><td>Platform Dependent</td><td>Platform Independent</td></tr><tr><td>Code execution</td><td>Direct</td><td>Direct</td><td>Executed by JVM (Java Virtual Machine)</td></tr><tr><td>Approach</td><td>Top-down approach</td><td>Bottom-up approach</td><td>Bottom-up approach</td></tr><tr><td>File generation</td><td>.exe files</td><td>.exe files</td><td>.class files</td></tr><tr><td>Pre-processor directives</td><td>Support header files (#include, #define)</td><td>Supported (#header, #define)</td><td>Use Packages (import)</td></tr><tr><td>keywords</td><td>Support 32 keywords</td><td>Supports 63 keywords</td><td>50 defined keywords</td></tr><tr><td>Datatypes (union, structure)</td><td>Supported</td><td>Supported</td><td>Not supported</td></tr><tr><td>Inheritance</td><td>No inheritance</td><td>Supported</td><td>Supported except Multiple inheritance</td></tr><tr><td>Overloading</td><td>No overloading</td><td>Support Function overloading (Polymorphism)</td><td>Operator overloading is not supported</td></tr><tr><td>Pointers</td><td>Supported</td><td>Supported</td><td>Not supported</td></tr><tr><td>Allocation</td><td>Use malloc, calloc</td><td>Use new, delete</td><td>Garbage collector</td></tr><tr><td>Exception Handling</td><td>Not supported</td><td>Supported</td><td>Supported</td></tr><tr><td>Templates</td><td>Not supported</td><td>Supported</td><td>Not supported</td></tr><tr><td>Destructors</td><td>No constructor neither destructor</td><td>Supported</td><td>Not supported</td></tr><tr><td>Multithreading/ Interfaces</td><td>Not supported</td><td>Not supported</td><td>Supported</td></tr><tr><td>Database connectivity</td><td>Not supported</td><td>Not supported</td><td>Supported</td></tr><tr><td>Storage Classes</td><td>Supported (auto, extern)</td><td>Supported (auto, extern)</td><td>Not supported</td></tr></table>	Metrics	C	C++	Java	Programming Paradigm	Procedural language	Object-Oriented Programming (OOP)	Pure Object Oriented	Origin	Based on assembly language	Based on C language	Based on C and C++	Developer	Dennis Ritchie in 1972	Bjarne Stroustrup in 1979	James Gosling in 1991	Translator	Compiler only	Compiler only	Interpreted language (Compiler + interpreter)	Platform Dependency	Platform Dependent	Platform Dependent	Platform Independent	Code execution	Direct	Direct	Executed by JVM (Java Virtual Machine)	Approach	Top-down approach	Bottom-up approach	Bottom-up approach	File generation	.exe files	.exe files	.class files	Pre-processor directives	Support header files (#include, #define)	Supported (#header, #define)	Use Packages (import)	keywords	Support 32 keywords	Supports 63 keywords	50 defined keywords	Datatypes (union, structure)	Supported	Supported	Not supported	Inheritance	No inheritance	Supported	Supported except Multiple inheritance	Overloading	No overloading	Support Function overloading (Polymorphism)	Operator overloading is not supported	Pointers	Supported	Supported	Not supported	Allocation	Use malloc, calloc	Use new, delete	Garbage collector	Exception Handling	Not supported	Supported	Supported	Templates	Not supported	Supported	Not supported	Destructors	No constructor neither destructor	Supported	Not supported	Multithreading/ Interfaces	Not supported	Not supported	Supported	Database connectivity	Not supported	Not supported	Supported	Storage Classes	Supported (auto, extern)	Supported (auto, extern)	Not supported	Unordered Associative Containers: implement unordered data structures that can be quickly searched: unordered_set, unordered_multiset, unordered_map, unordered_multimap	<pre>graph LR SF1[Source File] --> Compiler OF1[Object File] SF2[Source File] --> Compiler OF2[Object File] OF1 --> Linker EX[Executable] OF2 --> Linker EX EX --> OS PM[Program in Memory]</pre>	Vector provides • A sequence • Dynamically sized • Ability to add / remove anywhere • Random access • Bounds safety
Metrics	C	C++	Java																																																																																								
Programming Paradigm	Procedural language	Object-Oriented Programming (OOP)	Pure Object Oriented																																																																																								
Origin	Based on assembly language	Based on C language	Based on C and C++																																																																																								
Developer	Dennis Ritchie in 1972	Bjarne Stroustrup in 1979	James Gosling in 1991																																																																																								
Translator	Compiler only	Compiler only	Interpreted language (Compiler + interpreter)																																																																																								
Platform Dependency	Platform Dependent	Platform Dependent	Platform Independent																																																																																								
Code execution	Direct	Direct	Executed by JVM (Java Virtual Machine)																																																																																								
Approach	Top-down approach	Bottom-up approach	Bottom-up approach																																																																																								
File generation	.exe files	.exe files	.class files																																																																																								
Pre-processor directives	Support header files (#include, #define)	Supported (#header, #define)	Use Packages (import)																																																																																								
keywords	Support 32 keywords	Supports 63 keywords	50 defined keywords																																																																																								
Datatypes (union, structure)	Supported	Supported	Not supported																																																																																								
Inheritance	No inheritance	Supported	Supported except Multiple inheritance																																																																																								
Overloading	No overloading	Support Function overloading (Polymorphism)	Operator overloading is not supported																																																																																								
Pointers	Supported	Supported	Not supported																																																																																								
Allocation	Use malloc, calloc	Use new, delete	Garbage collector																																																																																								
Exception Handling	Not supported	Supported	Supported																																																																																								
Templates	Not supported	Supported	Not supported																																																																																								
Destructors	No constructor neither destructor	Supported	Not supported																																																																																								
Multithreading/ Interfaces	Not supported	Not supported	Supported																																																																																								
Database connectivity	Not supported	Not supported	Supported																																																																																								
Storage Classes	Supported (auto, extern)	Supported (auto, extern)	Not supported																																																																																								
<table><tr><td>Static • Memory is allocated at compile time • Memory is read/write • One copy alone. Not dynamic • Use the "static" keyword • Lifetime spans the entire execution of the program </td><td>Stack • Memory allocated for local variables of functions • Allocated automatically when entering the function • De-allocated automatically when you leave the function. • Each recursive invocation gets its own set of local variables! </td><td>Heap • Memory comes from for manual "on-the-fly" allocations • Two simple APIs: new and delete • The programmer is in charge for both allocation / deallocation • Lifetime of memory blocks is as long as they are not freed! </td><td>signal is a function passing an int and a pointer to a function passing an int returning nothing (void) returning a pointer to a function passing an int returning nothing</td></tr><tr><td><pre>fp is a pointer to a function passing an int and a pointer to float returning a pointer to a char char *(*fp)(int, float *);</pre></td><td><pre>* Default constructor: Called when no arguments provided * Custom constructor: Called with arguments to setup the instance * Copy constructor: Initializes an object using another object of the same class. The source object that is passed to the constructor will have its member fields copied into the new object. * Move constructor: The source object that is passed to the constructor will have its member fields moved into the new object.</pre></td><td>BIRTH • Release whatever resource is held by the instance (memory, files, network connections, etc.) • There is only one destructor. • can use the "virtual" keyword to handle polymorphic destruction. </td></tr><tr><td>Public: call class members are available to everyone. Data members and member functions can be accessed By other classes and functions too.</td><td>Private: accessed only by member function inside class Not allowed to be accessed directly by any object or funct Outside of class. Only mem function or friend functions Allowed to access private data members</td><td>Protected: similar to private, cant be Accessed from outside unless with friend. But protected class members can be Accessed by its subclasses</td></tr></table>	Static • Memory is allocated at compile time • Memory is read/write • One copy alone. Not dynamic • Use the "static" keyword • Lifetime spans the entire execution of the program	Stack • Memory allocated for local variables of functions • Allocated automatically when entering the function • De-allocated automatically when you leave the function. • Each recursive invocation gets its own set of local variables!	Heap • Memory comes from for manual "on-the-fly" allocations • Two simple APIs: new and delete • The programmer is in charge for both allocation / deallocation • Lifetime of memory blocks is as long as they are not freed!	signal is a function passing an int and a pointer to a function passing an int returning nothing (void) returning a pointer to a function passing an int returning nothing	<pre>fp is a pointer to a function passing an int and a pointer to float returning a pointer to a char char *(*fp)(int, float *);</pre>	<pre>* Default constructor: Called when no arguments provided * Custom constructor: Called with arguments to setup the instance * Copy constructor: Initializes an object using another object of the same class. The source object that is passed to the constructor will have its member fields copied into the new object. * Move constructor: The source object that is passed to the constructor will have its member fields moved into the new object.</pre>	BIRTH • Release whatever resource is held by the instance (memory, files, network connections, etc.) • There is only one destructor. • can use the "virtual" keyword to handle polymorphic destruction.	Public: call class members are available to everyone. Data members and member functions can be accessed By other classes and functions too.	Private: accessed only by member function inside class Not allowed to be accessed directly by any object or funct Outside of class. Only mem function or friend functions Allowed to access private data members	Protected: similar to private, cant be Accessed from outside unless with friend. But protected class members can be Accessed by its subclasses																																																																																	
Static • Memory is allocated at compile time • Memory is read/write • One copy alone. Not dynamic • Use the "static" keyword • Lifetime spans the entire execution of the program	Stack • Memory allocated for local variables of functions • Allocated automatically when entering the function • De-allocated automatically when you leave the function. • Each recursive invocation gets its own set of local variables!	Heap • Memory comes from for manual "on-the-fly" allocations • Two simple APIs: new and delete • The programmer is in charge for both allocation / deallocation • Lifetime of memory blocks is as long as they are not freed!	signal is a function passing an int and a pointer to a function passing an int returning nothing (void) returning a pointer to a function passing an int returning nothing																																																																																								
<pre>fp is a pointer to a function passing an int and a pointer to float returning a pointer to a char char *(*fp)(int, float *);</pre>	<pre>* Default constructor: Called when no arguments provided * Custom constructor: Called with arguments to setup the instance * Copy constructor: Initializes an object using another object of the same class. The source object that is passed to the constructor will have its member fields copied into the new object. * Move constructor: The source object that is passed to the constructor will have its member fields moved into the new object.</pre>	BIRTH • Release whatever resource is held by the instance (memory, files, network connections, etc.) • There is only one destructor. • can use the "virtual" keyword to handle polymorphic destruction.																																																																																									
Public: call class members are available to everyone. Data members and member functions can be accessed By other classes and functions too.	Private: accessed only by member function inside class Not allowed to be accessed directly by any object or funct Outside of class. Only mem function or friend functions Allowed to access private data members	Protected: similar to private, cant be Accessed from outside unless with friend. But protected class members can be Accessed by its subclasses																																																																																									

LONGEST COMMON PREFIX: std::string prefix = words[0]; for (string in words) { while (string doesn't start with prefix) { prefix = prefix.substr(0, prefix.length() - 1); } }	string stack; // BALANCED PARENTHESES CHECK for (char c in input) { if (opening bracket) push to stack; if (closing bracket) { if (stack empty OR not matching) return false; else pop from stack; } } return stack.empty();	String Permutation Check 1. Count characters in s1 2. Use window of s1.length() to check substrings of s2 3. Slide window and update counts s.substr(start, length) - Extract substring s.push_back(c) - Add character to end s.pop_back() - Remove last character s.back() - Get last character s.empty() - Check if string is empty s.length() or s.size() - Get string length	// EFFICIENTLY COUNT OCCURRENCES IN A SORTED ARRAY USING BINARY SEARCH std::vector<int> ECNumOccurrences(const std::vector<int>& arrayA, const std::vec std::vector<int> arrayC; for(int i = 0; i < arrayA.size(); i++) { // Count occurrences by finding the range where the value appears int count = std::upper_bound(arrayB.begin(), arrayB.end(), arrayA[i]) - std::lower_bound(arrayB.begin(), arrayB.end(), arrayA[i]); arrayC.push_back(count); } //ArrayA = {1, 3} //ArrayB = {1, 1, 2, 3, 6, 7, 10} return arrayC; //Result should be: {2, 1} // 1 appears twice, 3 appears once
---	---	--	---

SPACE COUNT FROM FILE #include <iostream> #include <fstream> using namespace std; int CountWords(char const *fileInput) { ifstream f(fileInput); if (!f.is_open()) { cout << "Bad file." << endl; return -1; } char CurrentCharacter; int SpaceCount = 0; int WordCount = 0; while(f.get(CurrentCharacter)) { if (CurrentCharacter == ' ') { SpaceCount++; } WordCount = WordCount + 1; } if (!WordCount) { return 0; } return SpaceCount + 1; }	FIND LONGEST COMMON PREFIX #include <vector> #include <string> using namespace std; string ECLongestPrefix(const vector<string> &words) { if (words.empty()) { return ""; } string prefix = words[0]; for(size_t i=0; i < words.size(); i++) { while(words[i].substr(0, prefix.length()) != prefix) { prefix = prefix.substr(0, prefix.length() - 1); if (prefix.empty()) { return ""; } } } return prefix; } // FIND INTEGER IN SET(can swap), RETURN BOOL bool ECFind(const set<int> &listNums, int x) { for(auto i : listNums) { // Iterate through set if (i == x) { // Check if current element matches targ return 1; } } return 0; // Not found	VALID PARENTHESIS CHECKER #include <string> #include <iostream> using namespace std; bool ECParenthesesCheck(const string& strInput) { string par; for (char c : strInput) { if (c == '(' c == '[') { par.push_back(c); } else if (c == ')') { if (par.empty() par.back() != '(') { return false; } par.pop_back(); } else if (c == ']') { if (par.empty() par.back() != '[') { return false; } par.pop_back(); } } return par.empty(); }	REMOVE MATCHING STRING POINTERS #include <vector> #include <string> using namespace std; void removeMatch(vector<string *> &string_pointers, string *to_remove) { for (int i=0; i < string_pointers.size(); i++) { if(string_pointers[i] == to_remove) { string_pointers.erase(string_pointers.begin() + i); i--; } } } void ECCountDistinct(const vector<int> &listNums, map<int,int> &mapDistinct) { for (auto i : listNums) { // For each number in the vector mapDistinct[i]++; // Increment its count in the map } //COUNT DISTINCT NUMBERS IN VECTOR } //Counts occurrences of each unique number in a vector and stores results in a map. int main() { const int arr[] = {1,3,5,3,3,5,2,3,1}; vector<int> vec(arr, arr + sizeof(arr) / sizeof(arr[0])); // Create vector from array map<int,int> mapDist; ECCountDistinct(vec, mapDist); // Count occurrences cout << "size of mapDist: " << mapDist.size() << " " << mapDist[1] << " " << mapDist[2] < cout << mapDist[3] << " " << mapDist[5] << endl; }
--	--	---	---

#include <vector> // GENERIC COMMON NUMBER FINDER #INCLUDE <Iostream> #include <string> #include <algorithm> using namespace std; template <typename T> bool ECCommonNumber(const vector<T> &listA, const vector<T> &listB, T &val) { for (auto i : listA) { // Iterate through first vector for (auto j : listB) { // Iterate through second vector if (i == j) { // Check if elements match val = i; // Store the common value return true; } } } return false; // No common elements found } template<class T> void ASSERT_EQ(T x, T y) { if(x == y) { cout << "Test passed: equal: " << x << " " << y << endl; } else { cout << "Test FAILED: equal: " << x << " " << y << endl; } //Finds the first common element } // between two vectors of any type using templates.	Generic Sum Calculator for Any Container and Type template <typename T, typename container> T ECSumList(const container &listNums) { T sum = T(); // Initialize sum with default constructor for(auto i : listNums) { // Iterate through container sum += i; // Add each element to sum } return sum; } #include <algorithm> template<class SrcContainer, class DestContainer> void ECCopy(const SrcContainer& src, DestContainer& dest) dest.clear(); // Empty destination container first for(const auto& i : src) { // Iterate through source containe dest.insert(dest.end(), i); // Insert each item at // the end of destination } SUM ARRAY ELEMENTS USING POINTERS void ECSumArray(const int *listNums, int size) { const int* p = listNums; // Create pointer to array int sum = 0; for(int i = 0; i != size; i++) { sum += p[i]; // can replace p with listNums } std::cout << sum << std::endl; }	C++ CASTING EXAMPLES // Cast int to char char ch1 = 'a'; int x1 = static_cast<int>(ch1); // or int(ch1) // Cast int pointer to double pointer (unsafe) int y1 = 10; double *py1 = reinterpret_cast<double*>(&y1); // Cast char pointer to int pointer (unsafe) char ch = 'a'; int *pch = reinterpret_cast<int*>(&ch); // Cast void pointer to int pointer int x4 = 20; void *px4 = &x4; int *px4i = static_cast<int*>(px4); // Cast const int to non-const int pointer // (dangerous) const int x5 = 0; int *px5 = const_cast<int*>(&x5); // In the function: modify const int through // non-const pointer void foo(const int *px) { int *pcx = const_cast<int*>(px); *pcx = 1; }	#include <iostream> using namespace std; string ECReverseInt(const string &strNumber) { string res; int n = strNumber.size(); int i = n - 1; // Skip trailing zeros while(i >= 0 && strNumber[i] == '0') { i--; } // REVERSE INT STR REMOV TRAILING 0s // Reverse the remaining digits for(i >= 0; i--) { res += strNumber[i]; } // Handle case of all zeros if(res.empty()) { res = "0"; } return res; } // TRIM STRING TO MAXIMUM 5 CHARACTERS void TrimString(std::string &x) { std::string res; for(unsigned int i=0; i<5 && i<x.size(); i++) { // mx 5 characters res += x[i]; // Add each character to result } x = res; // Assign trimmed result back to original string }
---	---	--	---

ARRAY OUT OF BOUNDS int main() //BUFFER OVERFLOW { int w = 0; char arr[] = "hello"; // 6 chars int i=8; arr[i] = 'T'; // writing at 8 cout << "arr = " << arr << endl; cout << "w = " << w << endl; return 0; }	REMOVE SPECIFIC VALUE FROM ARRAY void ECRemoveSpace(int nums[], int len, int val) { int count = 0; for(int i = 0; i != len; i++){ if(nums[i] != val){ nums[count] = nums[i]; count++; } } }	Insert Commas Between String Characters string ECStringSep(const string &str) { if(str.empty()) { // Handle empty string case return ""; } string res; for(auto i: str) { // Process each character res += i; // Add character res += ','; // Add comma separator } res.pop_back(); // Remove last comma return res; }	SWAP LETTER CASE std::string ECSwapCase(std::string str) { for(auto &a : str){ if(a >= 'a' && a <= 'z'){ a += 'A' - 'a'; } else if (a >= 'A' && a <= 'Z'){ a += 'a' - 'A'; } } return str;}	1. MEMORY & POINTERS TRUE: C++ does NOT perform bounds checking for arrays. TRUE: You must manually dealloc memory allocated with new. FALSE: delete can be used on a pointer allocated with malloc(). TRUE: nullptr is the modern replacement for NULL. FALSE: const_cast can safely remove const for modification. TRUE: A reference (&) must always be initialized when declared. FALSE: A reference can be reassigned to another variable. TRUE: A pointer can be reassigned to another address.
--	--	--	---	---

2. FUNCTIONS & CONST FALSE: Passing a vector by reference ensures it remains unchanged. TRUE: Using const prevents modification of a reference parameter. TRUE: Function overloading is based on different function signatures. FALSE: Function overriding can occur without virtual in the base class. TRUE: Default arguments are resolved at compile time. FALSE: static_cast can be used for safe downcasting of polymorphic types. TRUE: dynamic_cast only works on polymorphic types (with virtual functions).	3. OBJECT-ORIENTED PROGRAMMING & POLYMORPHISM TRUE: destructor should be virtual in base classes for proper cleanup. FALSE: pure virtual function must have an implementation in base class. TRUE: A class with a pure virtual function is an abstract class. FALSE: A derived class must override all virtual functions. TRUE: A final method cannot be overridden. FALSE: A class marked as final can still be inherited. 5. Compilation & Miscellaneous TRUE: The #ifndef preprocessor directive prevents multiple inclusions of a header file. FALSE: #define allows defining functions like in regular C++. TRUE: nullptr is the preferred way to represent a null pointer in modern C++. FALSE: The delete operator can be called on an object more than once. TRUE: static variables inside functs retain their values across funcn calls. FALSE: auto and decltype always deduce the same type.	4. STL & ITERATORS TRUE: std::vector elements are stored contiguously in memory. FALSE: std::list allows random access like std::vector. TRUE: Iterators provide a way to traverse containers safely. FALSE: Reverse iterators can be used on all STL containers. TRUE: std::set automatically keeps elements sorted. FALSE: std::unordered_map maintains elements in a sorted order. TRUE: std::map allows access to elements by key. SETS (std::set, std::unordered_set) Elements Are Always Unique → {1, 2, 2, 3} → {1, 2, 3} Fast Lookup with .count() → if (s.count(x)) (faster than .find()) Ordered (std::set) vs. Unordered (std::unordered_set) → O(log n) vs. O(1) avg insert/search Safe Erase While Iterating → for (auto it = s.begin(); it != s.end();) if (*it % 2 == 0) ? s.erase(it) : ++it; Find Min/Max → *s.begin(); (min); *s.rbegin(); (max) Convert Vector to Set (Remove Duplicates) → std::set<int> s(v.begin(), v.end());	VECTORS (std::vector) Pass by Reference to Avoid Copying → void func(std::vector<int>& v); Use .at() instead of [] for Bounds Checking → v.at(i); (safe) vs. v[i]; (unsafe) Use erase-remove to Delete While Iterating → v.erase(std::remove(v.begin(), v.end(), val), v.end()); Clearing a Vector Doesn't Free Memory → v.clear(); (keeps capacity) vs. std::vector<int>().swap(v); (frees) Sorting a Vector → std::sort(v.begin(), v.end()); (asc), std::sort(v.rbegin(), v.rend()); (desc) Finding an Element → if (std::find(v.begin(), v.end(), x) != v.end()) {} HEAP MEMORY ALLOCATION: double* arr = new double[10]; // using new or double* arr = (double*) malloc(10 * sizeof(double)); // Using malloc
---	--	---	---

